

RULE TUNING

EWA MAKOSA

The Linnaeus Centre for Bioinformatics
Department of Information Technology
UPPSALA UNIVERSITY
Uppsala, Sweden

EXE01/2005

Abstract

The aim of this thesis is to propose heuristic algorithms for rule tuning within rough set theory. The key functionality of the provided methods is determined by the ability to transform a generated set of rules into a smaller set, containing more general rules, with little loss of accuracy. This is available through algorithms of rule shortening and generalization.

Classical models based on decision rules induced from data often consist of large amounts of rules. Such large sets of rules cannot be efficiently processed. Moreover, too many rules may lead to over-fitting. Reduction in the number of rules is also important for experts analyzing the processed data.

The problem here is to develop methods for both reducing the number of rules and modification of rules resulting in more universal models. The desired procedure is to transform a set of rules into one of smaller, more feasible size whilst having as little effect on the classification quality as possible.

The heuristic algorithms of rule tuning were proposed and then implemented in the ROSETTA system. A number of tests were performed on large genomic data sets, with the analysis of these results incorporated into this report.

KEYWORDS: rule tuning, rough set theory

Acknowledgments

It was a great honor for me to work under supervision of Professor Jan Komorowski in the Linnaeus Center for Bioinformatics. I am grateful to Dr Hung Son Nguyen, who first introduced me to rough set theory and who came up with the idea for the rule pairwise generalization. I would like to thank Dr Michael Baldamus who took time and care to read this thesis for all his valuable remarks. Robin Andersson and Dr Torgeir R. Hvidsten were my guides to the ROSETTA project. I would like to thank Helena Strömbergsson for providing the datasets for the tests. Thanks to my dear parents, and my wonderful friends Marta Łuksza, Jakub Jurkiewicz and Marcin Kierczak for their support. And to Staś, for straightening my thoughts.

Uppsala, Sweden, June 26, 2005

EWA MAKOSA

CONTENTS

1	INTRODUCTION	1
1.1	Motivation	1
1.2	Related work	2
1.3	Overview	2
2	THEORETICAL BACKGROUND	3
2.1	Basic notions of rough set theory	3
2.2	Reducts	6
2.3	Decision rules	8
2.3.1	Numerical quantities of a decision rule	9
2.3.2	General decision rules	10
2.4	Measuring performance of rule-based classifiers	11
2.5	Over-fitting	12
2.6	Model transformations	13
3	RULE TUNING	15
3.1	Rule shortening	15
3.1.1	The rule shortening heuristic	16
3.2	Rule generalization	18
3.2.1	Rule generalization methods	18
3.2.2	Rule pairwise generalization	19
3.2.3	Rule group generalization	23
4	IMPLEMENTATION	29
4.1	The ROSETTA system	29
4.2	Implementation of proposed methods	30

4.2.1	Introducing general rules	30
4.2.2	Algorithms	30
5	RESULTS	33
5.1	Ligand-receptor binding affinity data	33
5.2	Tests	33
6	DISCUSSION	36
6.1	Rule tuning	36
6.2	Test result analysis	36
6.3	Shortening of rules	38
6.4	Method comparison	39
6.5	Future work	40
6.6	Conclusions	40

1 INTRODUCTION

1.1 Motivation

Machine learning deals with inducing models from data. To describe models their *predictive* and *descriptive* aspects are distinguished. Predictive models are simply models which classify new data objects automatically. *Descriptive* models are predictive models which are legible and interpretable by humans. These models can be seen as "white boxes", since their internal structure reveals inner mechanisms ruling the trends that might appear in data.

Rough set theory introduced by Zdzisław Pawlak [19] forms the theoretical basis for systems that discover propositional rules from data. In short, rough set theory deals with the approximation of sets that are difficult to describe with available knowledge. This could be due to lack of information or noise in the data.

Among many different techniques in data mining, rule-based models come as methods of choice for the sake of being both predictive and descriptive. Hence not only do they function well as classifiers of unseen cases, but they also allow comprehensive analysis of their structure in order to gain new insight into the underlying relationships buried in the data. Here we consider rule based models as collections of propositional rules of the form:

$$(a_1 = v_1) \wedge \dots \wedge (a_n = v_n) \Rightarrow (d = d^1) \vee \dots \vee (d = d^k)$$

Rules in a rule-based model are inherently descriptive and can be treated as representation of patterns appearing in data. A collection of rules can provide an insight into trends in data unless the set of rules becomes large and prohibitively complex [12]. Rule-based models that are too big loose interpretability. Oversized models often result from noisy data. In those cases the majority of the rules describe only a few examples from the training set rather than capture a general trend in the whole domain.

The described problem is common in bioinformatics. Biological experimental data may contain noise caused by inaccurate measurements. It is also often very expensive to provide a good training set of experimentally checked examples. Therefore the models are induced from vague and size limited data. Scientists in many cases are more interested in discovering the regularities in the processed data than in the prediction itself. Therefore it is crucial to provide them with a descriptive model of feasible size. In case of rule-based models, the less the rules and the stronger they are, the better.

This thesis deals with improving large rule-based models by transforming them so that they become smaller and that they better depict the mechanisms underlying

the data characteristics. The methods described here perform so called *rule pruning*, which aims at increasing the legibility of the rule-based model whilst retaining its valid component. Pruning transforms the model in such a way that it consists of a smaller set of important, strong rules whilst maintaining an acceptable predictive quality.

1.2 Related work

A well known rule pruning approach has been developed for improving rule-based models and studied in rough-set literature. Implementations are available both in the ROSETTA [10] and the RSES [5] systems. RSES system has both filtering and generalization methods available, (see [16] for a description of the application of these methods). A comprehensive study on rule filtering is given in [1], and applications of this method are described in [18]. A method performing rule generalization based on clustering of rules is described in [13]. Dynamic reducts [2], or any other kind of approximate reducts, covered in Chapter 2.2 can be considered as a method for rule pruning as it results in general rules and prevents the incorporation of noise-derived information in the induced model.

1.3 Overview

To introduce the theoretical framework of this thesis, basic notions of rough set theory, machine learning, and rule pruning are covered in Chapter 2. Methods of rule tuning are presented in Chapter 3, providing a comprehensive description of the proposed heuristics. A short report on implementation is given in Chapter 4. Chapter 5 describes the application of the proposed methods to biological data and reports the results. A closing summary is provided in Chapter 6.

2 THEORETICAL BACKGROUND

This chapter gives an introduction to rough set theory, a mathematical framework which provides tools for representing and gaining knowledge from often impure and ambiguous data. Rough set formalisms for representing and processing knowledge about data aim to discern between the objects and to classify them accordingly. The inner machinery of rough sets is boolean reasoning [6], which provides the synthesis of minimal and approximate definitions of concepts. The rough set-based approach is a method for dealing with incomplete, inconsistent and noisy data.

The notions introduced here are presented the way they are defined in the ROSETTA system. An introduction to rough set theory can be found in many publications, e.g. [11], [4].

2.1 Basic notions of rough set theory

Definition 2.1.1. An information system is a pair $I = (U, A)$, where U is a non-empty, finite set called *universe* and A is a non-empty, finite set of *attributes*. Elements of U are called *objects*.

Information about objects is given by a partial function $a : U \rightarrow V_a$ assigning values to attributes. Here V_a is called a *value set* of a .

A specific type of an information system called a decision table is considered.

Definition 2.1.2. A *decision table* is an information system of the form $I = (U, A \cup d)$, where $d \notin A$. Elements of A are called *conditions*, while d is called a *decision*.

For an example of a decision table please refer to table 2.1. Here U contains 18 objects with values concerning patients, $d = \text{Site of origin}$ and

$$A = \{\text{Gene1}, \text{Gene2}, \text{Gene3}, \text{Smoking}\}.$$

The column *Patient* is introduced only for convenience reasons, to refer to the consequent objects in the table in the examples that follow later in this chapter.

The objects in table 2.1 describe 18 patients with metastatic cancer with a known site of origin [7]. The data include the expression of three genes, where value -1 indicates that the gene was down-regulated, 0 that it remained unchanged and 1 that it was up-regulated compared to healthy tissue. Information on whether the patient smokes or not is also included.

The distinguished attribute d represents a classification of the objects from U . In machine learning decision tables are referred to as *training sets* of examples, with the

Table 2.1. A decision system: site of origin for metastatic cancer data

Patient	Gene1	Gene2	Gene3	Smoking	Site of origin
x_1	-1	-1	0	Yes	Lung
x_2	0	0	0	Yes	Lung
x_3	0	-1	1	No	Colon
x_4	0	0	0	Yes	Lung
x_5	0	-1	0	Yes	Lung
x_6	-1	-1	0	Yes	Lung
x_7	-1	1	0	No	Colon
x_8	-1	1	0	No	Colon
x_9	0	1	0	Yes	Colon
x_{10}	-1	-1	1	No	Lung
x_{11}	0	-1	0	Yes	Lung
x_{12}	0	-1	0	Yes	Lung
x_{13}	0	-1	1	No	Colon
x_{14}	0	1	1	No	Colon
x_{15}	-1	1	0	No	Colon
x_{16}	-1	-1	1	No	Colon
x_{17}	0	-1	0	Yes	Lung
x_{18}	0	-1	1	No	Lung

values of the attribute d describing classes that objects belong to. In the exemplary decision table the decision attribute is *Site of origin*.

Let $|(V_d)| = k$. Note that the decision d determines the partition $\{X_1, \dots, X_k\}$ of the universe U into k subsets.

Definition 2.1.3. For a value set $V_d = \{d_1, \dots, d_k\}$ there is a partition into k *decision classes* of objects of the form $X_i = \{x \in U : d(x) = d_i\}$, $1 \leq i \leq k$. Every decision class X_i constitutes a classification of objects in U determined by the decision d . For any $u \in U$ the decision class $\{x \in U : d(x) = d(u)\}$ is denoted $X(u)$.

In the exemplary data there are two decision classes:

$$X_{Colon} = \{x_3, x_7, x_8, x_9, x_{13}, x_{14}, x_{15}, x_{16}\}$$

$$X_{Lung} = \{x_1, x_2, x_4, x_5, x_6, x_{10}, x_{11}, x_{12}, x_{17}, x_{18}\}$$

For every set of attributes $B \subset A$, there exists an equivalence relation $IND_I(B)$ which equates two objects u and u' if they are indiscernible by attributes from B .

Definition 2.1.4. The B – *indiscernibility* relation is defined by:

$$IND_I(B) = \{(u, u') \in U \times U : \forall_{a \in B} a(u) = a(u')\}.$$

The equivalence classes obtained from $IND_I(B)$ are denoted $[x]_B$, $x \in U$.

For example, consider the equivalence class for object x_1 and subset of attributes $B = \{Gene1, Gene2\}$:

$$[x_1]_{\{Gene1, Gene2\}} = \{x_1, x_6, x_{10}, x_{16}\}.$$

The set of all equivalence classes for $\{Gene1, Gene2\}$ is equal to:

$$\begin{aligned} & \{\{x_1, x_6, x_{10}, x_{16}\}, \\ & \{x_2, x_4\}, \\ & \{x_3, x_5, x_{11}, x_{12}, x_{13}, x_{17}, x_{18}\}, \\ & \{x_7, x_8, x_{15}\}, \\ & \{x_9, x_{14}\}\}, \end{aligned}$$

whereas the set of all equivalence classes for $\{Gene1, Gene2, Gene3\}$ is equal to:

$$\begin{aligned} & \{\{x_1, x_6\}, \\ & \{x_2, x_4\}, \\ & \{x_3, x_{13}, x_{18}\}, \\ & \{x_5, x_{11}, x_{12}, x_{17}\}, \\ & \{x_7, x_8, x_{15}\}, \\ & \{x_9\}, \\ & \{x_{10}, x_{16}\}, \\ & \{x_{14}\}\}. \end{aligned}$$

An attribute $a \in B$ is *dispensable* in B if $IND_I(B) = IND_I(B \setminus \{a\})$, and *indispensable* otherwise. Set $B \subset A$ is *independent* on A if every attribute from B is indispensable in A , otherwise it is *dependent* on A .

The attribute *Smoking* is dispensable in the set of all attributes of the cancer origin data:

$$IND_I(\{Gene1, Gene2, Gene3, Smoking\}) = IND_I(\{Gene1, Gene2, Gene3\}).$$

In rough set theory concepts and their estimations are considered. A *concept* is represented by a chosen subset X of objects from the universe.

Let $B \subset A$. The sets $\underline{B}X = \{x : [x]_B \subset X\}$ and $\overline{B}X = \{x : [x]_B \cap X \neq \emptyset\}$

are called the *B-lower approximation* of X and the *B-upper approximation* of X respectively.

Let $B = \{Gene1, Gene2, Gene3\}$. An illustration of the notion of set approximations is given by estimating the set of patients where the origin of cancer is the lung.

$$\begin{aligned} \underline{B}X_{Lung} &= [x_1]_B \cup [x_2]_B \cup [x_5]_B \\ \overline{B}X_{Lung} &= [x_1]_B \cup [x_2]_B \cup [x_5]_B \cup [x_3]_B \cup [x_{10}]_B. \end{aligned}$$

Now for these examples the following relation holds:

$$\underline{B}X_{Lung} \subset X_{Lung} \subset \overline{B}X_{Lung}.$$

Definition 2.1.5. The set $\underline{B}X = \overline{B}X \setminus \underline{B}X$ is called the *B-boundary region* of X .

If $\underline{B}X = \emptyset$ then X is *exact* and if $\underline{B}X \neq \emptyset$ then X is *rough (inexact)*. The interpretation of $\underline{B}$ is that it represents the ambiguity in information about objects from X .

For the given examples the B-boundary region $\underline{B}X_{Lung}$ is equal to:

$$\underline{B}X_{Lung} = [x_3]_B \cup [x_{10}]_B.$$

Definition 2.1.6. If $\{X_1, \dots, X_k\}$ are decision classes of $\mathbf{I}$, the set $POS_B(d) = \underline{B}X_1 \cup \dots \cup \underline{B}X_k$ is called the *B-positive region* of $\mathbf{I}$.

The concept of patients with cancer originating in the colon, X_{Colon} , can be estimated using attributes in B :

$$\underline{B}X_{Colon} = [x_7]_B \cup [x_9]_B \cup [x_{14}]_B.$$

The positive region for the cancer origin data is defined by:

$$\underline{B}X_{Colon} \cup \underline{B}X_{Lung} = [x_7]_B \cup [x_9]_B \cup [x_{14}]_B \cup [x_1]_B \cup [x_2]_B \cup [x_5]_B.$$

Note that $[x_3]_B$ and $[x_{10}]_B$ are not contained in $POS_B(d)$, giving $|POS_B(d)| = 3 + 1 + 1 + 2 + 2 + 4 = 13$.

2.2 Reducts

For a good knowledge representation it is only needed to keep the minimal number of attributes that preserve the indiscernibility relation.

Definition 2.2.1. A *reduct* $\rho \subset A$ is a subset of the set of attributes A that is independent in A and $IND_{\mathbf{I}}(\rho) = IND_{\mathbf{I}}(A)$.

For example the set $\{Gene1, Gene2, Gene3\}$ is a reduct, since

$$IND_{\mathbf{I}}(\{Gene1, Gene2, Gene3\}) = IND_{\mathbf{I}}(A).$$

Definition 2.2.2. A *local reduct* $\rho(o) \subset A$ is a subset of A such that:

- $\forall o_i \in U \{d(o) \neq d(o_i) \Rightarrow \exists a_k \in \rho(o) : a_k(o) \neq a_k(o_i)\}$
- $\rho(o)$ is minimal with respect to inclusion.

A classical reduct, defined in 2.2.1 is sometimes referred to as a *global reduct*.

An *approximate reduct* is a set of attributes that discern objects in U to a certain degree. Informally, these attribute sets "almost" preserve the indiscernibility relation. From such reduct approximations one can generate decision rules that reveal probabilistic (non-deterministic) relationships between the set of conditions and the set of possible decisions.

Consider the subset of attributes $C \subset A$ in the information system $\mathbf{I} = (U, A \cup d)$.

Definition 2.2.3. The *degree of dependency* of d on C is defined by:

$$\gamma(C, d) = \frac{|POS_C(d)|}{|U|}.$$

For example, let $C = \{Gene1, Gene2\}$. Then,

$$\begin{aligned}\underline{C}X_{Colon} &= [x_7]_C \cup [x_9]_C \\ \underline{C}X_{Lung} &= [x_2]_C.\end{aligned}$$

Calculate the positive region for C :

$$POS_C(d) = \underline{C}X_{Colon} \cup \underline{C}X_{Lung} = [x_7]_C \cup [x_9]_C \cup [x_2]_C.$$

The degree of dependency of d on C is given by

$$\gamma(\{Gene1, Gene2\}, \{Site\ of\ origin\}) = \frac{3 + 2 + 2}{18} = \frac{7}{18},$$

while the degree of dependency of d on the reduct $\{Gene1, Gene2, Gene3\}$ equals

$$\gamma(\{Gene1, Gene2, Gene3\}, \{Site\ of\ origin\}) = \frac{13}{18}.$$

Definition 2.2.4. The *Error of reduct approximation* of the reduct $C \subset B$ is defined by

$$\varepsilon_{B,d}(C) = \frac{\gamma(B, d) - \gamma(C, d)}{\gamma(B, d)} = 1 - \frac{\gamma(C, d)}{\gamma(B, d)}$$

and measures how well C approximates B (relatively to the decision attribute).

For example, let $C = \{Gene1, Gene2\}$ and $B = \{Gene1, Gene2, Gene3\}$. Then

$$\varepsilon_{B,d}(C) = 1 - \frac{\gamma(C, d)}{\gamma(B, d)} = \frac{13}{18} - \frac{7}{18} = \frac{1}{3}.$$

There are several ways of computing reduct approximations:

- On the basis of a random sampling of the training data. The process of generating reducts is repeated for many different random samples, and the reducts that occur frequently (e.g. across at least 0.7 of these random samples) are considered to constitute the stable reduct approximations. The reducts that are computed on the basis of such repeated sampling are called *dynamic reducts* [2].
- Using a genetic algorithm. The population contains a number of possible reducts represented as bit strings of length $|A|$. The fitness function is obtained by computing the weighted sum of the reduct length and $\varepsilon_{B,d}(C)$.

2.3 Decision rules

Let $\mathbf{I} = (U, A \cup \{d\})$ be a decision table. Define V to be the set of all possible attribute values, $V = \bigcup\{V_a : a \in A \cup \{d\}\}$. There are atomic formulas over $B \subset A \cup \{d\}$ and V :

Definition 2.3.1. Expressions of the form $a = v$ where $v \in V_a$ are called *descriptors*.

Let B be a subset of conditions, $B \subset A$. Let V^B be a set of corresponding attribute values for B , i.e. $V^B = \bigcup\{V_a : a \in B\}$. Consider the set of formulas $P(B, V^B)$ over B and V^B which is the set of all descriptors over these sets closed with respect to conjunction. The meaning of $\psi \in P(B, V^B)$ in a decision table $\mathbf{I}$, that is the set of all objects in the universe U with the property ψ , is denoted as $\|\psi\|_{\mathbf{I}}$. $\|\psi\|_{\mathbf{I}}$ can be defined recursively:

$$\text{if } \psi \text{ is an atomic formula } a = v \text{ then } \|\psi\|_{\mathbf{I}} = \{x \in U : a(x) = v\} \quad (2.1)$$

$$\|\psi_1 \wedge \psi_2\|_{\mathbf{I}} = \|\psi_1\|_{\mathbf{I}} \cap \|\psi_2\|_{\mathbf{I}} \quad (2.2)$$

Let $a \in A \cup \{d\}$. The meaning $\|\varphi\|_{\mathbf{I}}$ of $\varphi \in S(a, V_a)$, where $S(a, V_a)$ is a set of all descriptors closed with respect to disjunction over a and V_a can be defined:

$$\text{if } \varphi \text{ is an atomic formula } a = v \text{ then } \|\varphi\|_{\mathbf{I}} = \{x \in U : a(x) = v\} \quad (2.3)$$

$$\|\varphi_1 \vee \varphi_2\|_{\mathbf{I}} = \|\varphi_1\|_{\mathbf{I}} \cup \|\varphi_2\|_{\mathbf{I}} \quad (2.4)$$

Definition 2.3.2. A *decision rule* for $\mathbf{I} = (U, A \cup d)$ is any expression of the form $\psi \Rightarrow \varphi$, where $\psi \in P(B, V^B)$ and $\varphi \in S(d, V_d)$ and $\|\psi\|_{\mathbf{I}} \neq \emptyset$. The formulas ψ and φ are referred to as the *antecedent* and the *conclusion* of the rule respectively.

Formulas over $S(d, V_d)$ are called *generalized decisions*.

The following exemplary decision rules constructed based on the reduct $\{Gene2, Smoking\}$ can be found:

$$\begin{aligned} r_1 &= (Gene2 = -1) \wedge (Smoking = Yes) \Rightarrow (Site\ of\ origin = Lung) \\ r_2 &= (Gene1 = -1) \wedge (Smoking = Yes) \Rightarrow (Site\ of\ origin = Lung) \\ r_3 &= (Gene2 = -1) \wedge (Smoking = No) \Rightarrow \\ &\quad (Site\ of\ origin = Lung) \vee (Site\ of\ origin = Colon). \end{aligned}$$

2.3.1 Numerical quantities of a decision rule

The number of the descriptors on the left hand side in a decision rule is called the *length* of this rule. A decision rule r_1 is *shorter* than r_2 if length of r_1 is smaller than the length of r_2 .

The decision rule $\psi \Rightarrow \varphi$ *matches* an object $u \in U$ iff $u \in \|\psi\|_I$.

The number of objects matched by a decision rule $\psi \Rightarrow \varphi$ is denoted $Match_I(\psi \Rightarrow \varphi)$ and is equal to the number of objects from U with the property ψ :

$$Match_I(\psi \Rightarrow \varphi) = \|\psi\|_I.$$

Definition 2.3.3. The *support* $Supp_I(\psi \Rightarrow \varphi)$ of a decision rule is the number of objects that match both the antecedent and the consequent:

$$Supp_I(\psi \Rightarrow \varphi) = \|\psi\|_I \cap \|\varphi\|_I.$$

This measure estimates the number of objects correctly predicted by the rule.

The decision rule $\psi \Rightarrow \varphi$ for I is *true* in I if $\|\psi\|_I \subset \|\varphi\|_I$. The decision rule $\psi \Rightarrow \varphi$ that is true in I is called *consistent* in I and *inconsistent* or *approximate* otherwise.

Definition 2.3.4. The *strength* of a decision rule r in I is defined as:

$$Strength_I(r) = \frac{Supp_I(r)}{|U|}.$$

A good estimate of consistency of the rule r in I is the ratio of a number of objects supporting the rule and the number of objects that match it. This measure is called *accuracy*.

Definition 2.3.5. The *accuracy* of the decision rule r in I is defined as:

$$Accuracy_I(r) = \frac{Supp_I(r)}{Match_I(r)}.$$

Note that the decision rule r is consistent in $\mathbf{I}$ iff $Accuracy_{\mathbf{I}}(r) = 1$.

For the decision rule r of the form $\psi \Rightarrow (d = d_1) \vee \dots \vee (d = d_k)$ accuracy is sometimes computed for each of the decision descriptors separately and given in the form of a vector v of length k , where

$$v_i = Accuracy_{\mathbf{I}}(\psi \Rightarrow (d = d_i)), 0 < i < k + 1$$

A straightforward algorithm calculating the rule's accuracy would iterate through all objects from the universe verifying which ones have the property ψ and which ones have both properties ψ and φ . This algorithm takes $O(n_I * m_I)$ time, where $n_I = |U|$ and $m_I = |A|$ are dimensions of the decision table $\mathbf{I}$.

The measure of the fraction of objects satisfying the conclusion of the rule that also satisfy the antecedent is called coverage:

Definition 2.3.6. The *coverage* of the decision rule $r = \psi \Rightarrow \varphi$ is defined as:

$$Coverage_{\mathbf{I}}(r) = \frac{Supp_{\mathbf{I}}(r)}{||\varphi||_{\mathbf{I}}}.$$

2.3.2 General decision rules

Decision rules introduced above are sometimes referred to as *classical rules* to distinguish them from an extended form of decision rules called *general decision rules*. The form

$$(a_1 = v_1) \wedge \dots \wedge (a_n = v_n) \Rightarrow (d = d^1) \vee \dots \vee (d = d^k)$$

is extended to

$$(a_1 \in S_a^1) \wedge \dots \wedge (a_n \in S_a^m) \Rightarrow (d = d^1) \vee \dots \vee (d = d^k)$$

or

$$[(a_1 = v_1^1) \vee (a_1 = v_1^2) \vee \dots] \wedge \dots \wedge [(a_n = v_n^1) \vee (a_n = v_n^2) \vee \dots] \Rightarrow (d = d^1) \vee \dots \vee (d = d^k)$$

Let $\mathbf{I} = (U, A \cup \{d\})$ be a decision table.

Definition 2.3.7. Let $a \in A$. Formulas over $S(a, V_a)$ are called *general descriptors*.

Consider the general descriptor $\phi \in S(a, V_a)$. Then ϕ is a formula of the form $(a = v_1) \vee \dots \vee (a = v_l)$ where $\{v_1, \dots, v_l\} = S_a \subset V_a$. The set S_a of values of the general descriptor will be called the *general attribute value set*.

Let $B \subset A$. The set of formulas $P_G(B, V^B)$ over B and V^B is considered, which is a set of all general descriptors over these sets closed with respect to conjunction.

The meaning $\|\phi\|_I$ of the general formula $\phi \in P_G(B, V^B)$ in the decision table I is defined by:

$$\text{if } \phi \in S(a, V_a) \text{ then its meaning is defined by (2.3) and (2.4)} \quad (2.5)$$

$$\|\phi_1 \wedge \phi_2\|_I = \|\phi_1\|_I \cap \|\phi_2\|_I \quad (2.6)$$

Definition 2.3.8. A *general decision rule* for I is any expression of the form $\phi \Rightarrow \varphi$, where $\phi \in P_G(B, V^B)$ and $\varphi \in S(d, V_d)$ and $\|\phi\|_I \neq \emptyset$.

Consider generalization of the following decision rules for the cancer data:

$$r_1 = (Gene2 = -1) \wedge (Smoking = Yes) \Rightarrow (Site \text{ of origin} = Lung)$$

$$r_2 = (Gene2 = -1) \wedge (Gene3 = 0) \Rightarrow (Site \text{ of origin} = Lung)$$

$$r_3 = (Gene2 = 0) \Rightarrow (Site \text{ of origin} = Lung)$$

$$r_4 = (Gene2 = -1) \wedge (Gene3 = 1) \Rightarrow (Site \text{ of origin} = Lung) \vee (Site \text{ of origin} = Colon)$$

$$r_5 = (Gene2 = -1) \wedge (Smoking = No) \Rightarrow (Site \text{ of origin} = Lung) \vee (Site \text{ of origin} = Colon)$$

$$r = (Gene2 = -1) \vee (Gene2 = 0) \Rightarrow (Site \text{ of origin} = Lung) \vee (Site \text{ of origin} = Colon)$$

Note that classical decision rules are a specific type of general decision rules. In the following chapters the general rules terminology is used. All the algorithms for generalization employ the general decision rules concept; if classical rules are input, their general attribute value sets for their descriptors have always only one element – the attribute value.

2.4 Measuring performance of rule-based classifiers

A *classifier* $\hat{d}$ over an information system $I = (U, A)$ is a function that maps an object $u \in U$ to the value of the decision attribute d in a decision system $I_T = (U_T, A \cup d)$, where $U_T \subset U$ is the data used to induce the rules of the classifier.

The classifier $\hat{d}$ is used to draw conclusions about objects from $U \setminus U_T$. Value of $\hat{d}(u)$ is a *prediction* of a decision value for u .

Measuring the classification ability of classifiers is practically impossible; the performance of a model on unseen cases cannot be calculated as the outcome on those cases is unknown.

Therefore one must estimate the performance of a classifier using measures on the available data. For classification ability tests, the input data U_T is divided into two subsets, called the *train* and the *test* datasets. The *test* dataset is used to simulate the unknown part of the whole universe U and contains the objects on which $\hat{d}$'s predictive performance will be tested.

A standard measure for comparing predictive performance in machine learning is a classifier's accuracy.

Definition 2.4.1. The *accuracy* of the classifier $\hat{d}$, relative to the decision system $\mathbf{I}$, is defined as the proportion of the number of objects that are correctly classified C to the total number of objects in the test set:

$$Accuracy_{\mathbf{I}}(\hat{d}) = \frac{C}{|U \setminus U_T|}.$$

Another way of measuring the classifier's performance is provided by the ROC (Relative Operating Characteristic) analysis. This approach originates in signal theory as a method for signal discrimination, and has been transferred to machine learning.

2.5 Over-fitting

In machine learning, a classifier is induced from a set of training examples, i.e. exemplary situations for which the desired output is known. In rough set theory the training data set is represented as a decision table. Since the training set is often only a small finite sample of the whole domain, it may have a biased representation of objects or not cover a part of the domain at all. Furthermore, in real-world applications, often some of the training data contain noise or other impurities.

When the rules are derived, the model is assumed to reach the state where it will be able to predict the correct output for other examples, thus generalizing to situations not presented during training.

Generalization is the key issue in developing a classifier. The question is, how well will the model predict decision values for cases that are not present in the training set? Similarly to other learning methods, for instance neural networks, rule-based models can suffer either from *under-fitting* or *over-fitting*.

- A model which is not sufficiently complex can fail to fully detect the patterns in a complex data set. Then the model is said to be under-fitted.
- If learning is continued too long or if training examples are rare or imperfect, the model may adjust to very specific random features of the training data that have no causal relation to the real trends in the data. This results in over-fitting.

In the case of rule-based models over-fitting leads to derivation of rules that are too specific and that incorporate noise, instead of revealing the picture of the whole domain. Less specific rules are more likely to better generalize on unseen cases. In the process of over-fitting, the performance on the training examples increases while the performance on the unseen data becomes worse. In [9] over-fitted classifier is defined as one for which there exists another classifier that performs worse on the training set, but better on the test set.

In both statistics and machine learning, in order to avoid over-fitting, it is necessary to use additional techniques. For the rule-based models, rule pruning comes as a remedy for over-fitting.

To give an example of how rule pruning transforms models to deal with over-fitting, an illustration is drawn in figure 2.5. The true concept that is estimated is a function drawn in black bold curve. The data given is an imperfect collection of points (a number of dots on the picture). An over-fitted model fits strictly to the data by learning a function that simply connects those points with straight lines. Generalization of such a model would introduce an "uncertainty" region (area filled with light grey in the figure), where the function has its values, in a shape more resembling the target function.

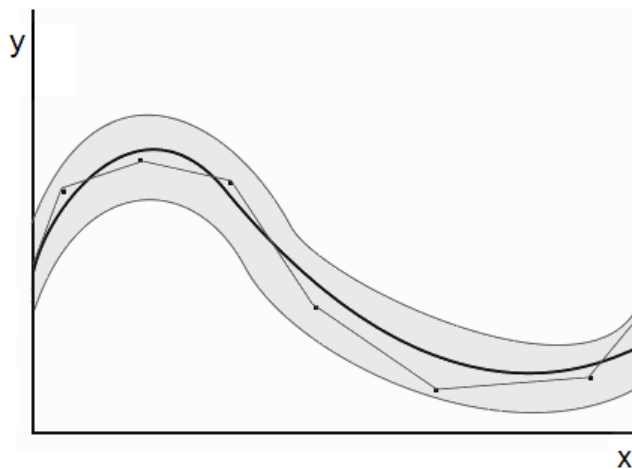


Figure 2.1. Over-fitting illustration

2.6 Model transformations

Rule pruning is a general method for dealing with over-fitting due to noise in the rule based models, and can also be used to extract small models representing the most general patterns in data.

There are two general categories of rule pruning:

1. Pruning done performed in the learning phase, i.e. while the rules are being induced, using heuristic stopping criteria for determining when to stop adding more literals to the rule or more rules to a rule set. One way of generalizing the model is to use the approximate reducts, see [24].
2. Pruning of the model obtained by the learning routine, done by applying heuristics to transform the induced model.

In classical approaches post-pruning is done by rule filtering. In this thesis a method of model transformation called rule tuning is proposed. Rule filtering attempts to find good sub-models by removing some rules. Rule tuning works by transforming rules in the model. As a side effect there is also a reduction of the number of rules. In this thesis the following methods for rule tuning are presented:

1. Rule shortening.
2. Rule generalization.
 - (a) Rule pairwise generalization.
 - (b) Rule group generalization.

3 RULE TUNING

To formalize the introduced terminology an operation τ_α on models is defined for rule-based classifiers:

Definition 3.0.1. Let Φ be a set of all general decision rules. Let $P(\Phi)$ be a set of all subsets of Φ . A τ_α – *transformation* of models is a function $\tau_\alpha : P(\Phi) \rightarrow P(\Phi)$, which for any model $m \in P(\Phi)$ returns $m' \in P(\Phi)$ of minimal size in terms of the number of rules, with the classification ability reduced at most to extent given by the parameter α .

The operation τ_α demands both the minimization of the model size and the maximization of its classification ability. Due to complexity of this problem and inability to verify the classification ability on the unseen cases, the proposed methods presented in this chapter are heuristics resulting in the approximation of the desired optimum τ_α .

The general schema of the proposed methods is to process sets of rules by transforming their elements. The classification ability is not computed, but estimated by the measures of particular rules. Instead of parameterizing the transformation τ with the model's accuracy, here the parameter α is the accuracy of each particular rule.

The rule tuning transformation of models operates on their elements. Each of the algorithms presented below provides a different implementation of the rule transformation. The proposed rule transformations are rule shortening and rule generalizations. Both result in the merging of rules, therefore reducing the cardinality of the input model.

3.1 Rule shortening

In real-world applications the training data often contains noise. Impurities in the data may result in too specific rules. These rules do not describe true dependencies in the knowledge domain and are usually long at the same time having low support. To deal with this problem, the rule shortening methods are applied.

Rule shortening can be simply defined as removing some descriptors from the rules. A rule with l descriptors sets the object in a l -dimensional space. Removal of descriptors allows free choice of values in their dimensions. The key problem is to choose the right subset of descriptors to constitute the left-hand side of the shortened rule. The search is targeted to obtain a rule which would perform better than the original one in classifying new objects from the unseen part of the domain.

Given a rule with the left hand side length l , a brute force search algorithm would

involve checking all the subsets of one rule's descriptors, yielding $2^l - 2$ possibilities. The proposed method is a heuristic, providing an approximation of the ideal solution.

After shortening some of the rules are merged, and therefore become stronger. Recall the decision rule example from Chapter 2.3 basing on the cancer data. For the two rules:

$$\begin{aligned} r_1 &= (Gene2 = -1) \wedge (Smoking = Yes) \Rightarrow (Site\ of\ origin = Lung) \\ Supp(r_1) &= 6, Accuracy(r_1) = 1 \\ r_2 &= (Gene1 = -1) \wedge (Smoking = Yes) \Rightarrow (Site\ of\ origin = Lung) \\ Supp(r_2) &= 2, Accuracy(r_2) = 1 \end{aligned}$$

shortening results in one universal rule

$$r = Smoking(Yes) \Rightarrow (Site\ of\ origin = Lung) \vee (Site\ of\ origin = Colon)$$

with $Supp(r) = 9$ and $Accuracy(r) = [0.888889, 0.111111]$.

Shortening of the optimal rule (generalized from the minimal reduct) always results in lowering of its accuracy. An acceptable level of the decrease of the accuracy α is the procedure's parameter. The pattern is clear if the accuracy for one of the decisions is significantly higher than the others, like in the example. A human interpretation of r would state that cancer in a patient who smokes is most likely to have originated in the lung.

Since removing descriptors of different rules may result in one identical rule, the last step of removing duplicate rules is necessary.

3.1.1 The rule shortening heuristic

For implementation in the ROSETTA system the rule shortening schema proposed in [3] was chosen. The procedure presented in heuristic algorithm 1 iteratively attempts to shorten each of the rules by removing their descriptors one by one until their accuracy reaches its lower boundary α . In a single iteration step, in a greedy manner, the best descriptor to be removed is found, i.e. the one that has the least effect on the accuracy.

Termination

The number of steps performed before the algorithm stops executing is defined by the outer loop iterating through the set of rules and the inner **while** loop.

There are two conditions of the **while** loop. Consider the second condition

$$r_{shortened}.length() > 1.$$

Heuristic 1 Rule shortening

Require: Set of rules $\mathfrak{R}$, threshold accuracy parameter $\alpha \in (0, 1]$ **Ensure:** Modified set of shortened rules

```

1: for all  $r \in \mathfrak{R}$  do
2:    $\alpha_{max} \leftarrow Accuracy(r)$ ;
3:    $r_{shortened} \leftarrow r$ ;
4:   while  $(\alpha_{max} > \alpha) \wedge (r_{shortened}.length() > 1)$  do
5:      $\alpha_{max} \leftarrow 0$ ;
6:     for all descriptors  $d$  of  $r_{shortened}$  do
7:        $r_{tmp} \leftarrow r_{shortened}$ ;
8:       Remove  $d$  from  $r_{tmp}$ ;
9:       Obtain accuracy of  $r_{tmp}$ ;
10:      if  $Accuracy(r_{tmp}) > \alpha_{max}$  then
11:         $\alpha_{max} \leftarrow Accuracy(r_{tmp})$ ;
12:         $d_{max} \leftarrow d$ ;
13:      end if
14:    end for
15:    if  $\alpha_{max} > \alpha$  then
16:      remove  $d_{max}$  from  $r_{shortened}$ ;
17:    end if
18:  end while
19:   $r \leftarrow r_{shortened}$ ;
20: end for
21: Remove duplicate rules from the rule set;

```

Assume the processed rule length is equal l . Since in every step of the inner loop one of the descriptors is removed, after at most the number of steps equal to $l - 1$ this condition is violated. This proves the algorithm stops, since the two conditions are joined by the operator $\wedge$.

Therefore the first condition,

$$\alpha_{max} > \alpha$$

is not necessary for the termination, but is needed since it is used to control the shortened rule's accuracy. This condition is dependent on the value of α_{max} . The value of α is set to a value in the $(0, 1]$ interval and does not change during the algorithm's execution. In each iteration step α_{max} is first assigned value 0. Then, if the condition of the **if** statement in line 12 holds, α_{max} is assigned the value of the accuracy of the shortened rule. Assume the condition of the **if** statement in line 12 always holds. Since shortening decreases the rule's accuracy, the value of α_{max} in each iteration step $i + 1$ is lower than in the iteration step i . If it is decreased below the value of α the iteration stops. If the condition of the **if** statement does not hold, α_{max} keeps its assigned value 0. Then, since $\alpha > 0$, $(\alpha_{max} > \alpha)$ does not hold.

Complexity

An estimate of the complexity of this algorithm needs to take into account obtaining the shortened rule's accuracy (line 10 of algorithm 1). Retrieving the rule's accuracy can be done in time $O(n_I \cdot m_I)$, where $n_I = |U|$ and $m_I = |A|$ (refer to Chapter 2.3). For each rule of length l this can be performed at most for all its descriptors, therefore in the worst case the number of steps is defined by the arithmetic series:

$$l + (l - 1) + \dots + 1 = \frac{l^2 + l}{2}$$

Therefore, having the longest rule in $\mathfrak{R}$ of length $m_{\mathfrak{R}}$ we estimate the length of each rule by $m_{\mathfrak{R}}$. For the set of rules of cardinality $n_{\mathfrak{R}}$ the complexity of algorithm 1 is estimated by $O(n_{\mathfrak{R}} \cdot n_I \cdot m_I \cdot m_{\mathfrak{R}}^2)$.

3.2 Rule generalization

Rule generalization transforms a rule-based model so that it consists of a number of general rules.

3.2.1 Rule generalization methods

Informally, generalization of a rule r with values taken from n other rules, proceeds by choosing a subset of r 's descriptors and then generalizing each of the descriptors

separately. To generalize one particular descriptor $a = S$, the corresponding descriptors in the remaining n rules are found, i.e. the ones that have the same attribute a . Values from their general attribute sets contribute to the new general attribute set S_g for a in the generalized rule. In other words, S_g is a subset of $S_G = S_1 \cup \dots \cup S_n$, where S_i is a generalized value set for a in rule r_i (if a does not belong to the set of attributes of r_i , then S_i is set to $\emptyset$).

The search space in the rule generalization problem is of a higher order than in rule shortening. First, the descriptors which are to be generalized need to be chosen and since any subset of the descriptors could be taken into account this already gives the complexity of the exhaustive search in the shortening problem, when descriptors were chosen to be removed.

In this case, when generalizing the rule r with the values taken from n other rules, for each of the chosen descriptors $a = S$, a brute force algorithm would also need to check all possible subsets of S_G .

Therefore heuristics need to be developed for choosing which rules to generalize together, then which of the descriptors to generalize and lastly, how to generalize them.

Two different approaches for rule generalization are proposed, rule pairwise and rule group generalization.

3.2.2 Rule pairwise generalization

Pairwise generalization proceeds by joining pairs of the rules on their descriptors. Let r_1, r_2 be a pair of the generalized rules and A_1, A_2 the corresponding attribute sets. The set of attributes for the resulting rule A_{gen} is the intersection of the attribute sets of the two:

$$A_{gen} = A_1 \cap A_2.$$

The general value sets of the general descriptor are obtained by generalization of the corresponding descriptors in both rules. For each attribute a and its general value sets S_1, S_2 in the rules r_1 and r_2 respectively, the new value set S_{gen} is a sum:

$$S_{gen} = S_1 \cup S_2.$$

The described process is performed by function 3 which returns the resulting general rule (line 7 in algorithm 2).

If $A_1 \neq A_2$, taking the intersection of the original attribute sets automatically shortens the rule. Assume r_1 is shorter than r_2 . To make sure not too many descriptors are taken away for the two rules to be generalized the inclusion of the two attribute sets is demanded, i.e.

$$A_1 \subset A_2.$$

This condition is checked by the $LHSSUBSET(r_2, r_1)$ function.

Rule pairwise generalization heuristic

Rule pairwise generalization schema is presented in heuristic algorithm 2.

The input set of rules $\mathfrak{R}$ is sorted (see line 2 in heuristic 2) descending by length. This is done to enforce generalization of the long rules first, i.e. the ones that are likely to result from noise in the data. The rationale for this choice is that pairwise generalization of two rules results in a rule of the length equal to the length of the shorter one.

The procedure continues iterating through rule indices in the sorted rule set trying to improve the model by generalizing a rule r_i of a particular index i with some other rule r_j .

In each iteration step the algorithm attempts to generalize r_i with one of the other rules which are further down on the rule list (i.e. which have the index j higher than i). Among these, the generalization which results in the least accuracy loss is chosen. If a good generalization with the rule r_g at the position g was found, the resulting general rule is placed at g , replacing r_g (to sustain the sorting order), while r_i is removed, reducing the number of rules in the model by one.

The algorithm attempts to generalize r_i with r_j if this pair is suitable to be generalized. Refer to function 4, which returns true if the two input rules are suitable for the generalization, for the conditions that need to be fulfilled. Due to the sorting of the rule list, it is assumed that r_j is shorter than r_i .

Termination

Algorithm 2 stops after iterating through the two nested loops. Denote the number of processed rules $n_{\mathfrak{R}} = |\mathfrak{R}|$. The outer **for** loop (lines 2 to 22) iterates through $n_{\mathfrak{R}}$ elements of the list L . For each i -th element of L , the inner loop (lines 4 to 17) performs $n_{\mathfrak{R}} - i - 1$ operations. Therefore the number of performed steps S after which the procedure stops is equal to the arithmetic series:

$$S = \frac{(n_{\mathfrak{R}} - 1) \cdot n_{\mathfrak{R}}}{2} \quad (3.1)$$

Complexity

To judge on the algorithm's complexity, consider a single iteration of the two nested for-loops of the algorithm. Let $m_{\mathfrak{R}}$ be the number of descriptors in the longest rule.

Heuristic 2 PAIRWISE GENERALIZATION

Require: set of rules $\mathfrak{R}$, threshold accuracy parameter α , rule attribute dissimilarity threshold δ

Ensure: a list of generalized rules

```

1:  $L \leftarrow$  Sort rules descending by length;
2: for all rules  $r_i \in L$  do
3:    $gen\_achieved \leftarrow false$ ;
4:   for all rules  $r_j \in L, j > i$  do
5:     if  $SUITABLE(r_i, r_j)$  then
6:        $r_{tmp} = GENERALIZE\_PAIR(r_i, r_j)$ 
7:       Obtain accuracy of  $r_{tmp}$ ;
8:       if  $Accuracy(r_{tmp}) > \alpha$  then
9:          $gen\_achieved \leftarrow true$ ;
10:        if  $Accuracy(r_{tmp}) > acc_{max}$  then
11:           $acc_{max} \leftarrow Accuracy(r_{tmp})$ ;
12:           $r_{gen} \leftarrow r_{tmp}$ ;
13:           $g \leftarrow j$ ;
14:        end if
15:      end if
16:    end if
17:  end for
18:  if  $gen\_achieved$  then
19:     $\mathfrak{R}.replace(g, r_{gen})$ ;
20:     $\mathfrak{R}.remove(i)$ ;
21:  end if
22: end for
23: Remove duplicate rules from  $L$ ;
24: return  $L$ ;

```

Function 3 *GENERALIZE_PAIR*(r_{from}, r_{to})

Require: two rules r_{from}, r_{to}

Ensure: if possible, general rule resulting from generalization of the r_{to} descriptors with the corresponding r_{from} values

```

1:  $r \leftarrow r_{to}$ 
2: for all descriptors  $d_{to}$  of  $r_{to}$  do
3:    $a_{to} \leftarrow d_{to}.getAttribute();$ 
4:   for all descriptors  $d_{from}$  of  $r_{from}$  do
5:     if  $d_{from}.getAttribute() = a_{to}$  then
6:       for all values  $v_{from}$  of  $d_{from}$  do
7:          $r.AddConditionValue(a_{to}, v_{from});$ 
8:       end for
9:     break;
10:  end if
11: end for
12: end for
13: return  $r$ ;

```

Function 4 *SUITABLE*(r_1, r_2)

Require: two rules r_1, r_2 , rule attribute dissimilarity threshold δ

Ensure: boolean value true if r_1 is suitable to have its descriptors generalized with correspondent r_2 values

```

1: if ( $r_1.attributeNo - r_2.attributeNo$ ) >  $\delta$  then
2:   return false;
3: end if
4: if ( $Accuracy(r_2) < \alpha$ ) then
5:   return false;
6: end if
7: if  $r_1.decision \neq r_2.decision$  then
8:   return false;
9: end if
10: return  $LHSSUBSET(r_2, r_1)$ ;

```

In step i of the iteration, the algorithm searches for the best generalization of the rule r_i with one of the remaining $n_{\mathfrak{R}} - i - 1$ rules. Assuming the worst scenario, all the checked rules will form a pair with r_i that is suitable for the generalization. In this case $n_{\mathfrak{R}} - i - 1$ "temporary" trial generalizations to the general rule r_{tmp} and the computation of its accuracy are performed. Both these operations contribute to the atomic algorithm operation o .

The trial generalization of the two rules is done by function 3, described above. This function iterates through both rules' descriptors, i.e. the attribute sets and their general value sets. Let s be the size of the largest of all the attribute value sets of the rules from $\mathfrak{R}$. Then the cost of function 3 is estimated by $O(m_{\mathfrak{R}}^2 \cdot s)$. Obtaining the rule's accuracy can be done in $O(n_I \cdot m_I)$, where $n_I = |U|$ and $m_I = |A|$ (refer to Chapter 2.3). Therefore the cost c of operation o is $O(m_{\mathfrak{R}}^2 \cdot s + n_I \cdot m_I)$.

Both operations *insert* and *replace* are linear, therefore can be estimated by $O(m_{\mathfrak{R}})$.

Thus the overall pessimistic cost is equal to the number of steps S (see equation 3.1) performed by the algorithm assuming o is performed in each of the steps:

$$\frac{n_{\mathfrak{R}}^2 \cdot c - n_{\mathfrak{R}} \cdot c}{2}$$

Thus the total complexity of the algorithm is estimated by

$$O(n_{\mathfrak{R}}^2 \cdot (m_{\mathfrak{R}}^2 \cdot s + n_I \cdot m_I)).$$

The algorithm's complexity is quadratic in both of the model's "dimensions", i.e. is the quadratic function of the number of rules $n_{\mathfrak{R}}$ and the largest rule's length $m_{\mathfrak{R}}$. The proposed methods are intended to deal with large rule sets in terms of $\mathfrak{R}$'s cardinality, where $m_{\mathfrak{R}}$ can be limited by some constant. The rule-based models, on which the algorithms were tested, had the maximum rule length equal to 7.

3.2.3 Rule group generalization

Rule group generalization instead of joining pairs of rules works with groups of similar rules. Generalization of a rule group g does not result in one general rule, but in $|g|$ rules. Each of the rules in the group is generalized separately, by selecting possible extensions of its general attribute value sets with values taken from the corresponding attribute value sets of the other rules in the group. The selection of the generalized descriptors proceeds in a greedy manner, choosing the best one for each step. The procedure for the rule group generalization is presented in function 7.

Rule group generalization heuristic

Rule group generalization begins with the initialization of rule groups (line 1 in heuristic 5). The groups are constructed in such a way that they contain similar rules that are likely to contribute most to good rule generalizations. The selection of rules to their groups is based on the assumption that merging of rules with the same general decision and overlapping sets of attributes prevents high drop in accuracy of the generated rule in comparison to the original ones.

Function 6 presents the schema for rule grouping. The input set of the rules $\mathfrak{R}$ is divided into a number of groups G of similar rules with the same generalized decision. This is done in two stages. First the rules are grouped according to their generalized decisions. Then, within each group g , the subsets of rules are found, which are ordered with respect to the attribute sets inclusion.

Heuristic algorithm 5 proceeds with generalizing each of the rules separately with the values from other rules in the same group.

The way the rules are generalized is presented in function 7. If possible, the generalization of the input rule r with chosen values of the corresponding descriptors of the other rules in its group g is performed. The greedy procedure is to iteratively pick the best descriptor for the generalization (having the least effect on the rule's accuracy), until the generalization of any of the remaining descriptors lowers the accuracy below the threshold α .

Heuristic 5 Rule group generalization

Require: set of rules $\mathfrak{R}$, threshold accuracy parameter α , coverage threshold σ

Ensure: a generalized set of rules $\mathfrak{R}$

```

1:  $G \leftarrow GROUP(\mathfrak{R})$ 
2: for all groups  $g$  in  $G$  do
3:   for all rules  $r_i$  in  $g$  do
4:      $GENERALIZE\_GROUP(r_i, g)$ 
5:   end for
6: end for
7:  $COVERAGE\_FILTER(\mathfrak{R}, \sigma)$ 

```

Termination

The termination needs to be verified for function 6 (line 1 of algorithm 5), the iterating loop (lines 2 to 6) with called function 7, and the coverage filtering performed in line 7.

The number of steps performed by group initializing function 6 is set by the number of iterations of the two nested **for** loops. The rule sets obtained in line 1 of the function are disjoint, therefore the iteration proceeds through the input set of rules

Function 6 $GROUP(\mathfrak{R})$

Require: set of rules $\mathfrak{R}$, threshold accuracy parameter α **Ensure:** set of rule groups G

```

1:  $S \leftarrow$  divide  $\mathfrak{R}$  into subsets according to their decision;  //generalize rules with
   the same decision
2: for all  $s \in S$  do
3:    $L \leftarrow$  sort rules in  $s$  based on length in descending order;
4:   for all  $r_i \in L$  do
5:      $l \leftarrow r_i$ ;  //get the longest rule to be grouped
6:     if  $\neg l.IsGrouped()$  then
7:        $g \leftarrow emptyGroup$ ;
8:        $g.add(l)$ ;  //find a chain of rules with subset order on attribute sets
9:       for all  $r_j \in L, j > i$  do
10:        if  $r_j.AttributeSet() \subseteq l.AttributeSet()$  then
11:           $g.add(r_j)$ ;
12:           $r_j.SetAsGrouped()$ ;
13:           $longest \leftarrow r_j$ ;
14:        end if
15:      end for
16:       $G.add(g)$ ;  //store the group
17:    end if
18:  end for
19: end for
20: return  $G$ 

```

Function 7 *GENERALIZE_GROUP*(r, g)**Require:** a base rule r , group of rules g , threshold accuracy parameter α **Ensure:** in-place generalization of r 's descriptors with chosen values of descriptors of rules in g

```

1:  $L_d \leftarrow$  a list of all  $r$ 's descriptors;
2:  $\alpha_{max} \leftarrow \text{Accuracy}(r)$ 
3: while ( $\alpha_{max} \geq \alpha$ )  $\wedge$  ( $L_d.length() > 0$ ) do
4:    $\alpha_{max} \leftarrow 0$ ;
   //Try to generalize each of the descriptors and evaluate those transformations
5:   for all  $d \in L_d$  do
6:      $r_{tmp} \leftarrow \text{GENERALIZE\_DESC}(r, g, d)$ ;
7:     Obtain accuracy of  $r_{tmp}$ ;
8:     if  $\text{Accuracy}(r_{tmp}) > \alpha_{max}$  then
9:        $\alpha_{max} \leftarrow \text{Accuracy}(r_{tmp})$ ;
10:       $d_{max} \leftarrow d$ ;
11:    end if
12:  end for
13:  if  $\alpha_{max} \geq \alpha$  then
14:     $L_d.remove(d_{max})$ ;
15:     $r \leftarrow \text{GENERALIZE\_DESC}(r, g, d_{max})$ ;
16:  end if
17: end while

```

$\mathfrak{R}$. The inner loop is executed only if the processed rule has not been assigned to any of the groups yet. Thus the number of steps performed before the function stops executing is in the pessimistic case equal to $\frac{(n_{\mathfrak{R}}-1) \cdot n_{\mathfrak{R}}}{2}$ and on average $n_{\mathfrak{R}}$.

The inner **for** loop of function 7 (lines 5 to 11) retrieves α_{max} . If $\alpha_{max} < \alpha$ the **while** loop (lines 3 to 16) condition does not hold and the algorithm stops executing. Assume $\alpha_{max} < \alpha$ never occurs. Then the condition at line 12 of the function always holds and one of the elements of list L_d is removed. Then after the number of iterations of the **while** loop equal to the length of L_d , $L_d.length() = 0$ and the constraint ($L_d.length() > 0$) does not hold.

Since the rule groups returned by function 6 are disjoint, algorithm 5 stops after the $m_{\mathfrak{R}}$ steps needed to iterate through the set of rules.

The analysis of the coverage filtering algorithm is covered at the end of this chapter.

Complexity

The complexity of the group initialization step (function 6) is a quadratic function of the number of rules $n_{\mathfrak{R}}$. First, each of the rules is assigned to one of the subsets of $\mathfrak{R}$ according to its decision, which is done in linear time. Then within these subsets, after sorting which takes $O(n_{\mathfrak{R}} \cdot \log(n_{\mathfrak{R}}))$, for each rule r_i a linearly ordered set of

rules is constructed to form a group of rules g , with a total order defined by the inclusion of attribute sets. The set inclusion is checked with $O(m_{\mathfrak{R}}^2)$ steps, where $m_{\mathfrak{R}}$ is the number of descriptors in the longest rule in $\mathfrak{R}$. Thus the construction of g is done in the pessimistic time $O(n_{\mathfrak{R}}^2 \cdot m_{\mathfrak{R}}^2)$ (on average $O(n_{\mathfrak{R}} \cdot m_{\mathfrak{R}}^2)$, since the set inclusion is checked only on those rules which were not grouped previously). That gives the estimation of the total pessimistic cost of function 6, equal to $O(n_{\mathfrak{R}}^2 \cdot m_{\mathfrak{R}}^2)$.

Following in heuristic 5 is the generalization of each of the rules with values from the rules from its group. In each iteration step i of function 7 one of the descriptors to be generalized is chosen among $l - i$ possible ones, which have not been generalized yet. The cost of the *GENERAZLIZE_DESC*(r, g, d) function is proportional to the number of rules multiplied by the size of value sets. Here the general value set of the descriptor d is being extended with values taken from value sets of all the corresponding descriptors from the rules in g . Therefore the complexity of *GENERALIZE_DESC*(r, g, d) is estimated by $O(n_{\mathfrak{R}} \cdot s)$, where s is the size of the largest of all attribute value sets of the rules from $\mathfrak{R}$. After each of those tested generalizations, accuracy of the resulting rules is computed. This can be done in $O(n_I \cdot m_I)$ time, where $n_I = |U|$ and $m_I = |A|$ (as described in Chapter 2.3). In the pessimistic case the number of descriptor generalizations performed in function 7 is equal to the arithmetic series:

$$\frac{l^2 + l}{2}.$$

Therefore, having the longest rule in $\mathfrak{R}$ of length $m_{\mathfrak{R}}$ the complexity of function 7 is $O(m_{\mathfrak{R}}^2 \cdot (n_{\mathfrak{R}} \cdot s + n_I \cdot m_I))$.

Having a call to function 7 for all of the rules in $\mathfrak{R}$, the total complexity is equal to $O(m_{\mathfrak{R}}^2 \cdot n_{\mathfrak{R}} \cdot (n_{\mathfrak{R}} \cdot s + n_I \cdot m_I))$.

Coverage filtering

Since the rule generalization procedure allows the same rules to contribute to generalization of the other ones a number of times, it may result in an information bias. If some rules provide their values for the generalization process more often than other ones, the objects used to build these rules have more impact on the generalized model than they used to have on the original one. To get rid of this effect, coverage-based filtering needs to be applied. The minimal coverage of a data set of objects with a coverage threshold σ is a minimal set of rules that match each of the objects at least σ times.

The filtering procedure presented in function 8 finds the upper approximation of the minimal coverage by removing some rules from the input rule set $\mathfrak{R}$. The heuristic is based on the assumption that the strongest rules are preferred to form the minimal coverage set. The heuristic algorithm proposed here does not search for the minimal solution for efficiency reasons. First, in the initialization step (line 2) all rules are

marked as "unused" (i.e. not belonging to the resulting coverage rule set). For each object, σ strongest rules that cover it are found and marked to join the resulting set (marked as "used" in the algorithm). The remaining rules which were not used in the covering process are filtered away.

Complexity

First, the rules are sorted, which takes $O(n_{\mathfrak{R}} \cdot \log(n_{\mathfrak{R}}))$ time. Then for all of n_I objects the set of rules is scanned. The removal of the unused rules is a linear operation and takes $O(n_{\mathfrak{R}})$. Therefore the complexity of this function is estimated by

$$O(n_{\mathfrak{R}} \cdot \log(n_{\mathfrak{R}}) + n_I \cdot n_{\mathfrak{R}})$$

Function 8 Coverage filtering

Require: Set of rules $\mathfrak{R}$, universe U , coverage threshold σ

Ensure: Coverage-based filtering of $\mathfrak{R}$

- 1: $L \leftarrow$ Sort rules in $\mathfrak{R}$ descending according to their support;
 - 2: Mark all rules as unused;
 - 3: **for all** $u \in U$ **do**
 - 4: $c \leftarrow 0$;
 - 5: **for all** $r \in L$ **do**
 - 6: **if** r matches u **then**
 - 7: mark r as used;
 - 8: $u \leftarrow u + 1$;
 - 9: **if** $c = \sigma$ **then**
 - 10: break;
 - 11: **end if**
 - 12: **end if**
 - 13: **end for**
 - 14: **end for**
 - 15: Remove all unused rules from $\mathfrak{R}$
-

4 IMPLEMENTATION

4.1 The ROSETTA system

Rule tuning algorithms were proposed and implemented in the ROSETTA system to extend its functionality.

ROSETTA is a toolkit for analyzing tabular data within the framework of rough set theory. It is designed to support the overall data mining and knowledge discovery process. The system allows initial browsing and preprocessing of data, computation of minimal attribute sets, generation of decision rules or descriptive patterns and their validation and analysis.

ROSETTA is implemented in the C++ programming language, and the design of the application is divided into a front-end and a kernel.

The front-end constitutes a graphical user interface (GUI) with data-navigational abilities emphasized, reflecting the contents of the kernel. The GUI is object-oriented. All manipulable objects are represented as individual GUI items, each with their own set of context-sensitive menus.

The computational kernel is also available as a command-line program, suitable for being invoked from other applications.

See [17] for a general presentation of the ROSETTA design. The kernel is a C++ class library. The most basic classes are the *Structure* and the *Algorithm* class:

- *Structure* classes represent different types of data. Deriving from it is class *Structures*, which represents a set or more complex collections of *Structure* objects.
- The *Algorithm* class is a base class for all algorithmic classes. Objects of those classes alter, create or process data structures. An *Algorithm* object takes a *Structure* object as input to its *Apply* method, and returns a *Structure* object if the application succeeded. The output structure may be the input structural object (modified or not), or a new structure of some type. All algorithms installed in the kernel are automatically available in the front-end.

An example of a class deriving from the class *Structure* is the class *Rule*. A collection of rules is implemented as class *Rules* extending *Structures*. Applying a learning algorithm (object of the *Algorithm* class) to a decision table (object of the *DecisionTable* class, deriving from the *Structure*) produces a set of rules (class *Rules*).

To prevent memory leaks and for implementation convenience the ROSETTA class

library has class template *Handle* implemented. Intuitively the *Handle* provides a smart pointer to class objects.

4.2 Implementation of proposed methods

4.2.1 Introducing general rules

Since for general rules the representation is extended, an implementation of classes dealing with rules in the ROSETTA system had to be provided. The main new class schema is as follows: the class *GeneralRule* deriving from *Rule* was added, as well as the class *GeneralRules* which extends the class *Rules*. An object of the class *GeneralRules* represents a collection of general rules. The class *GeneralRule* implements all the methods of the *Rule* class that needed to be overloaded for the sake of the new rule's representation, which in place of an attribute value introduces a general attribute value set. The data structure used to implement the general value set is the *STL* library *Set* class [25].

All the rule representation functionality that is made available by the ROSETTA system is adjusted for usage of general rules. Methods dealing with rule representation include:

- different views defined for rules, including rule statistics and String format.
- saving and loading of a rule object
- export to plain, Prolog, C++, and XML formats

4.2.2 Algorithms

Proposed algorithms as presented in Chapter 3 were implemented as classes deriving from the *Algorithm* class and optimized for efficiency.

Figure 4.2.2 presents a class hierarchy diagram for the main introduced classes. The base class added is the abstract *RuleTuner* class, with methods shared by all deriving classes. Its *Apply* method calls the *Tune* method which is overloaded in the subclasses. The *Tune* method corresponds to the τ_α transformation defined in Chapter 3. All of the algorithms should be applied to a set of rules and return a set of general rules (deriving from *Structure*).

The *RuleShortener* class derives directly from the *RuleTuner* class. For each of the rules the *Shorten* method is called, performing shortening of one rule.

Both classes implementing generalization heuristics, i.e. the *RulePairwiseGeneralizer* class and the *RuleGroupGeneralizer* class derive from the *RuleGeneralizer* class.

The rule pairwise generalization in the implementation has pre-grouping according to decision introduced, similarly to group generalization.

The *GeneralizedDecision* class representing generalized decision in ROSETTA was wrapped in *DecisionSet* class for the purpose of the tuning implementation. *DecisionSet* provides methods used in the rule grouping process, e.g. comparator methods.

The following type definitions were introduced for the generalization classes:

- *RuleGroup*: a vector of rule handles
- *DecisionSubsets*: a map, with key class *DecisionSet* and values of type *RuleGroup*

The *Discerner* class is a utility class for algorithms that need a discernibility predicate on a "per attribute" basis.

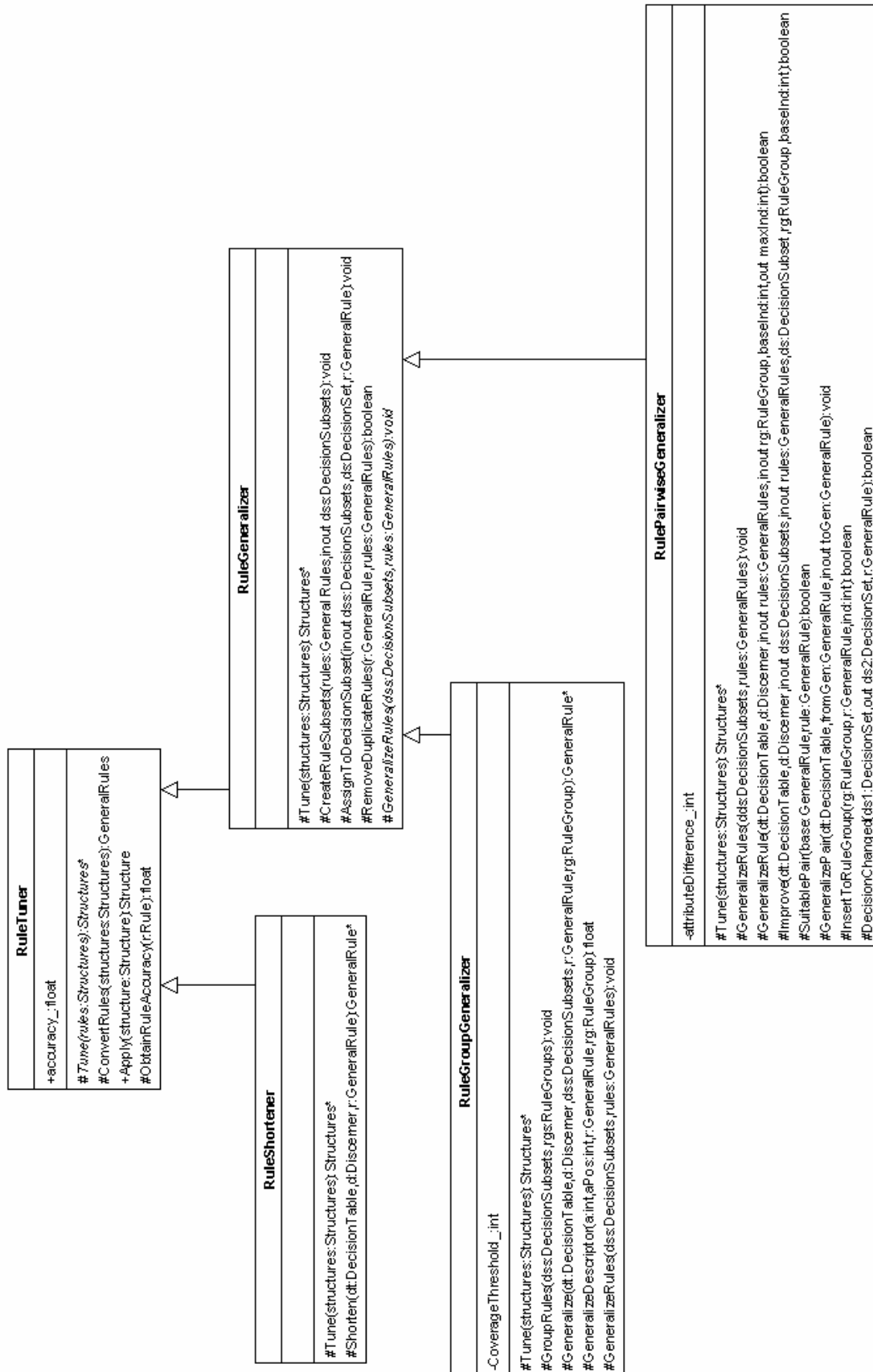


Figure 4.1. Class diagram

5 RESULTS

Rule tuning methods have been applied to proteochemometrics data [27] in which descriptors of receptors and ligands are associated with binding affinity, analyzed previously in a number of studies. Prusis et.al. [21], [20] applies the Partial Least Squares method [8]. In [26] the decision rule model was applied to the datasets with previously discretized binding affinity values.

The molecular recognition target in this problem is to find a combination of ligand-receptor sites resulting in the highest binding.

In this thesis the described data served as a training set to induce rule-based models on which the developed methods were tested.

5.1 Ligand-receptor binding affinity data

The first dataset (referred to as dataset *A*) contains 40 MC receptor-ligand complexes. Each object of the dataset is a description of the composition of MC receptor chimeras and peptide ligands together with the receptor-ligand binding affinity. The binding affinities are given as pKi which is the negative logarithm of the inhibition constant Ki. The pKi values are taken from [14]. The dataset has 6 nominal attributes, each having 2 possible values.

The second dataset (dataset *B*) is composed of 60 receptor-ligand complexes. Each object in the dataset describes the composition of MC receptor chimeras and six linear and cyclic peptide ligands. The receptors are described by the same descriptors as the first dataset. The binding affinity pKi values taken from the study made by [23]. The dataset has 8 nominal condition attributes, where 7 of the attributes have 2 possible values and one of them has 3 possible values.

5.2 Tests

There were so called *train-test* tests performed. Since the input data was ill-defined, the initial step in all of the tests was preprocessing of the data resulting in multiplying number of training objects. For the given decision tables so called *difference tables* were obtained, as described in [15].

For the purpose of this thesis a decision table transformation is denoted as Δ . From the given decision table $\mathbf{I} = (U, A \cup \{d\})$, where $A = \{a_1, \dots, a_k\}$, the new difference

table $I^* = (U^*, A^* \cup \{d^*\})$ is constructed as follows:

$$\begin{aligned}
 U^* &= U \times U = \{(x, y) : x, y \in U\} \\
 A^* &= \{a_j^* : U \times U \rightarrow V_{a_j} \times V_{a_j} \text{ for } j = 1, \dots, k\} \\
 a_j^*(x, y) &= (a_j(x), a_j(y)) \text{ for any pair of objects } x, y \in U \\
 d^* &: U \times U \rightarrow \{-1, 0, 1\} \\
 d^*(x, y) &= \theta_\varepsilon(x, y) \text{ for any pair of objects } x, y \in U
 \end{aligned}$$

Results of the tests performed are provided below. Measures are given for original models (where tuning method is given as "None") and for all rule tuning transformations that were performed. Transformations were repeated with different settings of the parameter α . In all tests of rule pairwise generalization rule attribute dissimilarity threshold δ parameter was set to 1. Rule group generalization had the coverage threshold parameter σ set to 2.

In the train-test setup, the datasets given were splitted into the train and test subsets, in proportion 70%:30% respectively. On both the train and test datasets the Δ transformation was performed, resulting in new decision tables.

For the input dataset A of size 40, the original train dataset A_{Train} had 28 elements, and original test set A_{Test} 12. Datasets $\Delta(A_{Train})$ and $\Delta(A_{Test})$ resulting from the Δ transformations contained 784 and 144 elements respectively.

For the input dataset B of size 60, the original train dataset B_{Train} had 42 elements, and the original test set B_{Test} 18. After applying the Δ transformation to these datasets $\Delta(B_{Train})$ and $\delta(B_{Test})$ were obtained, where $|\Delta(B_{Train})| = 1764$ and $|\Delta(B_{Test})| = 324$.

Rule-based models were induced in the ROSETTA system on $\Delta(A_{Train})$ and $\Delta(B_{Train})$ using two different algorithms, Johnson's and SavGenetic Reducer. The proposed methods were tested on the resulting rule models, by calculating accuracy of prediction for the elements int the test sets $\Delta(A_{Test})$ and $\Delta(B_{Test})$. Results of this test are presented in table 5.1. Columns of this table refer to rule models obtained by the two reducing algorithms Johnson's Reducer (referred to as Johnson's) and SavGenetic Reducer (refereed to as SavGenetic) on the given training sets obtained from datasets A and B.

Table 5.2 presents the sizes of the rule models before and after the tested transformations. The general trend is that the lower the α parameter is set for transformations, the smaller the resulting models are.

Table 5.1. Train-test experiment results achieved using different parameter settings.

Rule tuning method	Threshold α	Dataset A		Dataset B	
		Johnson's	SavGenetic	Johnson's	SavGenetic
None	-	79.86%	84.72%	69.75%	78.39%
Rule shortening	0.9	81.25%	83.33%	74.07%	79.01%
	0.95	82.63%	86.11%	73.14%	80.24%
Rule pairwise generalization	0.9	75.69%	72.22%	73.45%	78.70%
	0.95	76.38%	79.86%	72.22%	80.86%
	1	77.08%	80.55%	69.44%	75.61%
Rule group generalization	0.9	75%	79.86%	76.54%	81.48%
	0.95	78.47%	80.55%	75.92%	84.87%
	1	77.08%	81.94%	74.38%	82.71%

Table 5.2. Number of elements in reduced models in the train-test schema.

Rule tuning method	Threshold α	Dataset A		Dataset B	
		Johnson's	SavGenetic	Johnson's	SavGenetic
None	-	269	523	573	1583
Rule shortening	0.9	223	429	460	1140
	0.95	249	493	524	1459
Rule pairwise generalization	0.9	172	293	336	872
	0.95	172	302	344	883
	1	177	305	344	892
Rule group generalization	0.9	142	235	327	286
	0.95	183	288	391	441
	1	195	317	455	671

6 DISCUSSION

6.1 Rule tuning

The general problem that is challenged in this thesis is best described as improving large rule-based models trained on noisy, ambiguous data by transforming them with as little loss of their classification ability as possible.

This demand was formalized in the notion of τ_α transformation of rule-based classifiers, which both aims at minimization of the size of a model and maximization of its classification performance.

The proposed heuristic methods do not give the desired optimal solution. This is due to efficiency reasons and to the impossibility of retrieving the true value of a classifier's performance, which can only be estimated by measures on the train set.

One difficulty when using the rule tuning algorithms is setting the accuracy threshold so that the pruned model indeed performs statistically better on the unknown cases. The only solution is to adjust the parameter experimentally. The threshold strongly depends on the amount of noise in the data – the more disturbed it is, the lower the threshold should be set. Experimental results on exemplary data [3] indicate that the preferred values of α lay in the $(0.7, 1.0)$ interval. In tests performed for the purpose of this thesis a value of α set between 0.9 and 1 gave the best results.

The results of the performed tests indicate that all the proposed methods significantly reduce the size of the transformed models. Loss of accuracy is controlled by the parameter α . In the proposed methods α is not a limit of the accuracy of the classifier itself, but the accuracy of the rules that it contains. Still, tests measuring performance of the transformed classifiers show that manipulating accuracy of particular rules allows one to control the accuracy loss of the whole model. In most of the examples rule tuning decreased accuracy only by a few percent. Moreover, in a number of cases accuracy of the transformed models increased. The ability of improving classifiers performance by rule tuning is due to a reduction of over-fitting in a model.

6.2 Test result analysis

For both of the datasets and both reducing algorithms there were better accuracy measures achieved by one of the tested methods. For the smaller dataset rule shortening proved to be most effective, whereas for the larger one it was rule group generalization.

As shown in table 5.1 in all cases the resulting model was smaller than the original

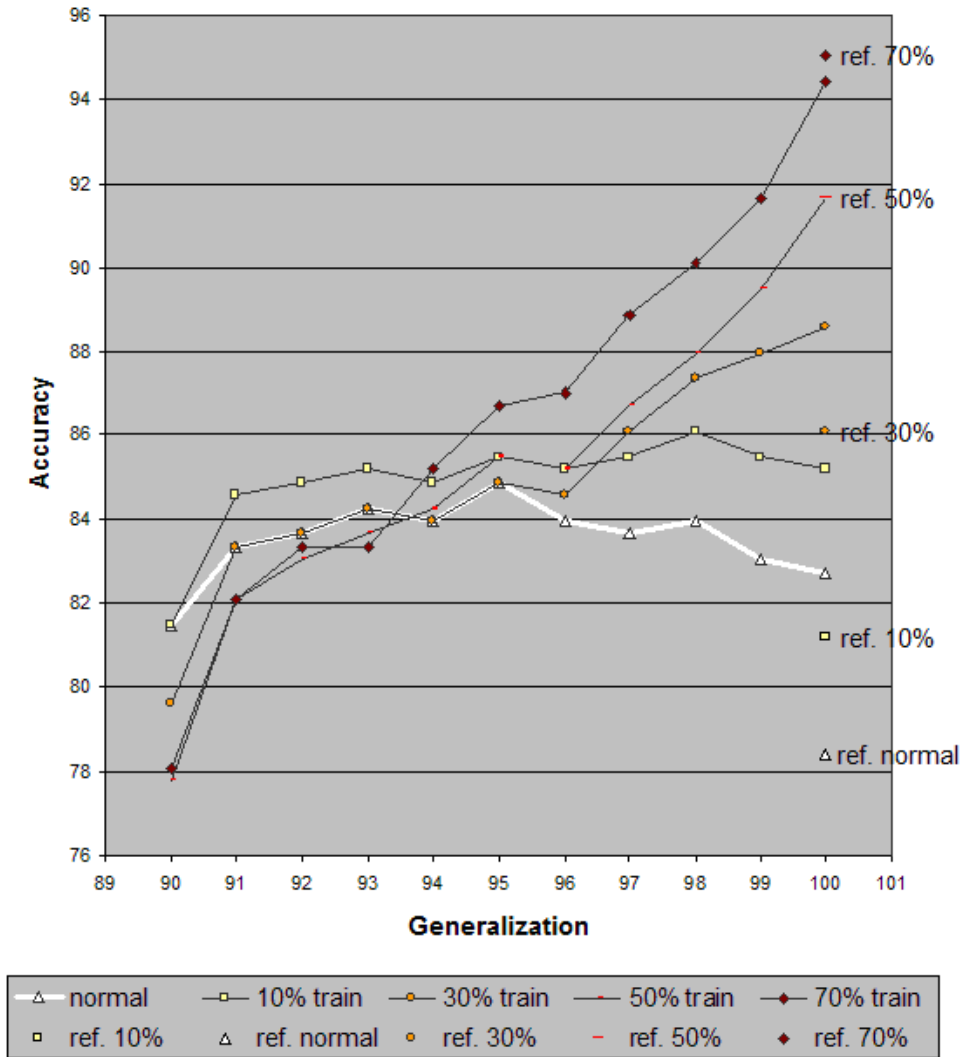


Figure 6.1. Comparison of accuracy measurements (Y axis) for rule group generalization with different values of parameter α (X axis) between tests performed on test datasets with different amounts of training examples included.

one. The lower the parameter α is set, the more significant reduction of the number of rules.

An experiment was performed, which tests whether rule tuning eliminates overfitting. Recall (see Chapter 2.5) that the more over-fitted the model is, the more its performance on the training examples increases and the worse its performance on the unseen data becomes. Diagram 6.1 presents comparison of performance of generalized models on test datasets with different content of training examples. It is to show how generalization changes classification ability on the unseen versus the known cases.

All of the models measured were trained on the $\Delta(B_{Train})$ dataset, described in

Chapter 5. The different plots correspond to different amounts of training examples introduced in the test set $\Delta(B_{Train})$. The data series labeled as *normal* is a plot of accuracy measures for tests performed on the original $\Delta(B_{Train})$ set, while the other plots *10% train*, *30% train*, *50% train*, *70% train* had 10%, 30%, 50%, and 70% of the original objects replaced with the training ones, respectively.

When tested on the "normal" test dataset, the generalized model got poor results when α was set too high, but for an α of around 0.95 transformed models performed better than for α near 1. Therefore a characteristic shape of the plot is obtained, having a peak for one value of α , and then descending values of accuracy in both of the X axis directions (higher and lower α values). The decreasing values of accuracy for α between 0.95 and 1 show that the more over-fitted the model is (i.e. less general), the worse its performance on the unseen data becomes. This regularity reveals how generalization changes performance of over-fitted models on the unseen cases and suggests how α should be set. If α is too low, the tuning transformation is too aggressive with resulting models lacking important knowledge. On the other hand performance of classifiers transformed by a tuning method with α set too high is close to the original model's since it does not introduce any improvements.

The more training examples introduced in the test set, the more plots for the generalized models differ from the *normal* plot. Instead of having a peak accuracy value for a certain α , the accuracy values increase with increasing α . The higher content of the training examples in the test set is, the stronger this tendency becomes and the less the plot resembles the *normal* one. Therefore the general observation is that when testing on the known cases, the fewer the changes were introduced in the model (by setting α high), the better the accuracy it obtained. This also proves that generalization reduces over-fitting – the more over-fitted (i.e. less general) the model is, the more its performance on the training examples increases.

6.3 Shortening of rules

Removing descriptors of different rules can result in the same shorter rule (see example in Chapter 3.1). This simple idea forms the basis for the rule shortening algorithm, which shortens rules merging repeating ones, leading to a model containing more general rules with high support. This approach follows the *Minimum Description Length Principle (MDLP)* [22], adapted to rough sets.

Since long rules are often caused by noise in data, the two remaining heuristics also aim to reduce the number of long rules. This can be interpreted as optimization in both "dimensions" of the model.

Rule pairwise generalization transforms long rules first. Since the rule pairwise generalization always results in a rule shorter or equal to the original one, in this process most often the long rules are replaced by shorter ones.

Transformation of rules in rule group generalization, described in Chapter 3.2, does

not remove the descriptors from the transformed rule itself. Reducing the number of long rules in the model is taken care of by applying coverage filtering. Since rules selected to form the coverage set are chosen based on their support, the longest ones will be omitted. This is due to the fact that long rules resulting from noise in data have low support.

6.4 Method comparison

The advantage of rule generalization over shortening is foremost due to the new representation of rules. The strength of classical rules is limited by their form. Allowing usage of disjunction in attribute values allows to represent the same information covered by several classical rules in a single general rule. This is why the generalizing τ_α transformation reduces the number of rules even for parameter α set to 1.

Rule shortening in general does not provide anything more than approximate reducts would, apart from the fact that there is more control on the resulting classifier's performance.

All the proposed methods proved to fulfill one of the most important conditions that rule tuning is supposed to meet: to improve the model's legibility. It is not only that they all result in smaller models, which are easier to be investigated by humans. Shorter rules put less specific constraints on the data and therefore depict more general patterns. Classifiers transformed by rule generalization methods result in general rules, which are more descriptive than classical ones. It is interesting to biologically interpret rules in the generalized models of the tested data. For example it was found that in some of the general rules, the value set of a particular attribute gathered rules of similar biological characteristics.

Therefore generalization methods should be more helpful when dealing with large rule sets, however they are much more computationally demanding than the shortening method.

Complexity of the generalization problem is one of the drawbacks for proposing an accurate algorithm searching for the optimal τ_α . As a part of this project a genetic algorithm was implemented that searched for minimal coverage of objects with a set of general rules. A population was constituted of a number of rule sets - organisms. A mutation in such an organism was performed by removal of randomly chosen rules. Crossing-over was defined as generalization of rules between organisms. The fitness function was the sum of the number of objects covered at least σ times and the proportional reduction of the number of rules. Tests showed that the described genetic algorithm was not time efficient. In many experiments it achieved solutions close to the rule group generalization, but took much longer time. As every optimization algorithm it converges to the optimal solution with time and how long it takes is very much dependent on a number of parameters such as crossover and mutation rate and stop condition parameters. Tests proved that the

described procedure was extremely sensitive to changes in these parameters, which is an obstacle for users.

6.5 Future work

It should be stated that the methods proposed in this thesis are only three of many possible heuristics. There are several aspects where different solutions could be investigated in rule generalization, i.e.

- Selection of subsets of rules to be generalized together. The objective here is to find groups of rules that are similar as regards some similarity measure. In this thesis the criterion was the same generalized decision and overlapping sets of attributes.
- Selection of descriptors to be processed. In rule pairwise generalization the common set of attributes for the two rules generalized is chosen and descriptors for these attributes contribute to the general ones. In rule group generalization selection of descriptors proceeds in a greedy manner, picking the one that has the least affect on accuracy.
- For the given attribute, finding the optimal subset of attribute values from the contributing rules could be considered. Both the methods proposed in this thesis having chosen descriptors to be generalized from a number of rules compute the general value set as a sum of the contributing sets of values.

Both in rule shortening and generalization, different classifier performance measures could be incorporated, for instance ROC analysis.

Both generalization algorithms inspire interesting formal analysis. In future studies, theoretical scientific efforts should be exerted in investigating the properties of proposed algorithms. Generalization of rules and its usage in the transformation of models could be formalized.

6.6 Conclusions

Methods for rule shortening and rule pairwise and group generalization presented in this thesis proved to implement rule tuning transformation of models. Performed tests show that tuned classifiers consist of a lower number of more general rules, which increases their legibility with little loss of accuracy. Moreover, the results obtained encourage the use of different rule tuning methods to find a transformation that not only results in a reduction of the model's size, but also increases its classification ability.

Therefore rule tuning is a method of choice when dealing with large models and with over-fitting.

Bibliography

- [1] ÅGOTNES, T. Filtering large propositional rule sets while retaining classifier performance. Master's thesis, Norwegian University of Science and Technology, February 1999.
- [2] BAZAN, J. A comparison of dynamic and non-dynamic rough set methods for extracting laws from decision tables. In *Rough Sets in Knowledge Discovery 1: Methodology and Applications* (Heidelberg, Germany, January 1989), A. S. L. Polkowski, Ed., Physica-Verlag.
- [3] BAZAN, J. *Metody wnioskowań aproksymacyjnych dla syntezy algorytmów decyzyjnych*. PhD thesis, Wydział Matematyki, Informatyki i Mechaniki Uniwersytetu Warszawskiego, Warszawa, Polska, 1998.
- [4] BAZAN, J. G., NGUYEN, H. S., NGUYEN, S. H., SYNAK, P., AND WRÓBLEWSKI, J. *Rough set algorithms in classification problem*. Physica-Verlag GmbH, Heidelberg, Germany, Germany, 2000.
- [5] BAZAN, J. G., AND SZCZUKA, M. S. Rses and rseslib - a collection of tools for rough set computations. In *Rough Sets and Current Trends in Computing* (2000), W. Ziarko and Y. Y. Yao, Eds., vol. 2005 of *Lecture Notes in Computer Science*, Springer, pp. 106–113.
- [6] BROWN, F. M. *Boolean Reasoning: The Logic of Boolean Equations*. Kluwer Academic Publishers, Dordrecht, 1990.
- [7] DENNIS, J. L., HVIDSTEN, T. R., WIT, E. C., KOMOROWSKI, J., BELL, A. K., DOWNIE, I., MOONEY, J., VERBEKE, C., BELLAMY, C., KEITH, W. N., AND OIEN, K. A. Markers of adenocarcinoma characteristic of the site of origin: Development of a diagnostic algorithm. *Clinical Cancer Research* 11, 10 (May 15 2005), 3766–3772.
- [8] GELADI, P., AND KOWALSKI, B. Partial least-square regression: A tutorial. *Anal Chim Acta* 185 (1986), 1–17.
- [9] HVIDSTEN, T. R. *Predicting Function of Genes and Proteins from Sequence, Structure and Expression Data*. PhD thesis, Uppsala University, Uppsala, Sweden, 2004.
- [10] KOMOROWSKI, J., ØHRN, A., AND SKOWRON, A. *Case studies: Public domain, multiple mining tasks systems: ROSETTA rough sets*. Oxford University Press, Inc., New York, NY, USA, 2002.
- [11] KOMOROWSKI, J., POLKOWSKI, L., AND SKOWRON, A. *Rough sets: a tutorial*, 1998.

- [12] LÆGREID, A., HVIDSTEN, T., MIDELFART, H., KOMOROWSKI, J., AND SANDVIK, A. Predicting gene ontology biological process from temporal gene expression patterns. *Genome Research* 13, 5 (2003), 965–979.
- [13] MIKOŁAJCZYK, M. Reducing number of decision rules by joining. In *TSCTC '02: Proceedings of the Third International Conference on Rough Sets and Current Trends in Computing* (London, UK, 2002), Springer-Verlag, pp. 425–432.
- [14] MUCUNIECE, R., MUTULE, I., MUTULIS, F., PRUSIS, P., SZARDENINGS, M., AND WIKBERG, J. Detection of regions in the mc_1 receptor of importance for the selectivity of the mc_1 receptor super-selective ms04/mso5 peptides. *Biochim Biophys Acta* 1544, 1–2 (Jan 2001), 278–282.
- [15] NGUYEN, H. S., ŁUKSZA, M., MAKOSA, E., AND KOMOROWSKI, H. J. An approach to mining data with continuous decision values. In *New Trends in Intelligent Information Processing and Web Mining (IIPWM'05), series "Advances in Soft Computing"* (Gdańsk, Poland, June 13-16 2005), Springer Verlag, pp. 653–662.
- [16] NGUYEN, H. S., AND NGUYEN, S. H. Analysis of stulong data by rough set exploration system (rses). In *ECML/PKDD-2003 Discovery Challenge Workshop* (2003), P. Berka, Ed., pp. 71–82.
- [17] ØHRN, A. Rosetta technical reference manual. Tech. rep., Norwegian University of Science and Technology (NTNU), Department of Computer and Information Science, Trondheim, Norway, 2000.
- [18] ØHRN, A., OHNO-MACHADO, L., AND ROWLAND, T. Building manageable rough set classifiers.
- [19] PAWLAK, Z. Rough sets. *Int. J. Comp. Inf. Sci.* 11 (1982), 341–56.
- [20] PRUSIS, P., LUNDSTEDT, T., AND WIKBERG, J. Proteo-chemometrics analysis of msh peptide binding to melanocortin receptors. *Protein Eng.* 15, 4 (Apr 2002), 305–311.
- [21] PRUSIS, P., MUCUNIECE, R., ANDERSSON, P., POST, C., LUNDSTEDT, T., AND WIKBERG, J. Pls modeling of chimeric ms04/msh-peptide and mc1/mc3-receptor interactions reveals a novel method for the analysis of ligand-receptor interactions. *Biochim Biophys Acta* 1544, 1–2 (Jan 2001), 350–357.
- [22] RISSANEN, J. Minimum-description-length principle. In *Encyclopedia of Statistical Sciences*. Wiley (1985), S. Kotz and N. Johnson, Eds., pp. 523–527.
- [23] SHIOTH, H., YOOK, R., MUCANIECE, R., WIKBERG, J., AND SZARDENINGS, M. Chimeric melanocortin mc1 and mc3 receptors: identification of domains participating in binding of melanocyte-stimulating hormone peptides. *Mol Pharmacol* 54, 1 (1998), 154–161.

- [24] ŚLĄZAK, D. Approximate reducts in decision tables. In *IPMU'96* (Granada, Spain, 1996), pp. 1159–1164.
- [25] STEPANOV, A. A., AND LEE, M. The Standard Template Library. Tech. Rep. X3J16/94-0095, WG21/N0482, 1994.
- [26] STRÖMBERGSSON, H., PRUSIS, P., MIDELFART, H., WIKBERG, J. E. S., AND KOMOROWSKI, H. J. Proteochemometrics modeling of receptor-ligand interactions using rough sets. In *German Conference on Bioinformatics* (2004), pp. 85–94.
- [27] WIKBERG, J., MARIS, L., AND PETERIS, P. *Proteochemometrics: A tool for modelling the molecular interaction space*. Weinheim: Wiley-VCH, 2004.